



LLMOps & MLOps for Agentic AI

A comprehensive operational guide for production AI agent systems

AGENTIC AI BUILDER EXPERT BOOTCAMP

BATCH 4.0 | APRIL 2026

Table of Contents

01

LLMOps vs MLOps

Key differences across dimensions

03

Prompt Management

Versioning and registry architecture

05

Observability & Tracing

LangSmith integration and key metrics

02

Agentic AI Lifecycle

Six-phase operations framework

04

Model Selection & Evaluation

Criteria, metrics, and pipelines

06

Cost, Guardrails & More

Safety, CI/CD, monitoring, incident response

What is LLMOps vs MLOps?

MLOps and LLMOps share the same goal — reliable AI in production — but differ significantly in workflow, tooling, and challenges.

Dimension	Traditional MLOps	LLMOps	Agentic AI Ops
Core artifact	Trained model weights	Prompts + model config	Graphs + prompts + tools
Training	Custom training pipelines	Fine-tuning / RLHF (rare)	Rarely trains; orchestrates
Data dependency	Training data critical	In-context data (RAG)	RAG + tool outputs + state
Evaluation	Metrics (F1, accuracy)	LLM-as-judge, human eval	End-to-end task completion
Cost driver	GPU training compute	API token usage	Multi-call chains compound
Failure modes	Model drift, data skew	Hallucination, refusal	Tool failures, infinite loops, state corruption

The Agentic AI Operations Lifecycle

Agentic AI operations follow a continuous lifecycle that extends traditional MLOps with agent-specific stages.



Prompt Management and Versioning

Prompts are the primary control interface for LLM behavior. **Treat them as code with full version control.**

Prompt Registry Architecture

```
class PromptRegistry:
    def register(self, name, template,
                 metadata=None):
        version_hash = hashlib.sha256(
            template.encode()
        ).hexdigest()[:8]
        entry = {
            "name": name,
            "version": version_hash,
            "template": template,
            "metadata": metadata or {},
            "created_at": datetime.utcnow()
                .isoformat(),
        }
        return entry
```

Best Practices

- **Version every change** — even a comma can alter behavior
- **Tag prompts** with model, temperature, max_tokens, use case
- **Use parameterized templates** — avoid hardcoding values
- **Store eval results per version** — link to benchmark scores
- **Use LangSmith Hub** for team collaboration and A/B testing

Model Selection and Evaluation

Selection Framework

Criteria	Weight	Example Metrics
Task accuracy	High	Pass rate on eval suite, F1
Latency	High	P50, P95, P99 response times
Cost per call	Medium	Token pricing, calls per task
Context window	Medium	128K vs 200K for RAG agents
Tool calling quality	High	% correct tool selection
Safety & alignment	High	Refusal rate, harmful output rate

Evaluation Pipeline

```
def accuracy_evaluator(run, example):
    predicted = run.outputs["response"]
    expected = example.outputs["expected"]
    return {
        "score": 1.0 if predicted.strip()
            == expected.strip() else 0.0,
        "key": "exact_match"
    }

results = evaluate(
    lambda inputs: agent.invoke(inputs),
    data="agent-eval-dataset",
    evaluators=[accuracy_evaluator],
    experiment_prefix="gpt-4o-v2"
)
```

CHAPTER 5

Observability and Tracing

For agentic AI, tracing is **not optional**. Every agent call involves multiple LLM invocations, tool calls, and state transitions. Without tracing, debugging is nearly impossible.



Key Observability Metrics

Metric	Description	Alert Threshold	Priority
Token usage/request	Total input + output tokens	> 50K tokens	High
Latency P95	95th percentile response time	> 30 seconds	High
Error rate	% of requests returning errors	> 2%	Critical
Tool failure rate	% of tool calls that fail	> 5%	High
Loop detection	Agent revisiting same node 3+ times	Any occurrence	High
Cost per request	Dollar cost across all LLM calls	> \$0.50	Medium
Hallucination rate	% of responses with factual errors	> 3% (sampled)	High

📄 LangSmith automatically traces all LLM calls, tool invocations, graph state transitions, and checkpoints when `LANGCHAIN_TRACING_V2=true` is set.

Cost Management

Agentic AI **compounds LLM costs** — a single user request can trigger 5–20+ LLM calls. Cost management is critical.



Model Cascading

Use cheaper models (GPT-4o-mini, Claude Haiku) for simple tasks; expensive models for complex reasoning



Caching

Cache identical prompt+input combos. Use semantic caching for similar queries



Prompt Optimization

Shorter prompts = fewer input tokens. Remove redundant instructions



Recursion Limits

Always set max iterations to prevent runaway loops and early termination for clear-cut cases



Batching

Group evaluations into batch API calls — 50% cheaper on most providers



Budget Alerts

Set per-project and per-user spending limits with automatic cutoff

Guardrails and Safety

Production agents need **multiple layers of safety guardrails** to prevent harmful, incorrect, or expensive outputs.

Guardrail Layers (Code)

```
def input_guard(user_message):  
    if detect_pii(user_message):  
        return False  
    if detect_injection(user_message):  
        return False  
    if not is_allowed_topic(user_message):  
        return False  
    return True  
  
def output_guard(agent_response):  
    response = redact_pii(agent_response)  
    response = verify_facts(response)  
    if is_toxic(response):  
        return "I cannot provide that."  
    return response
```

NeMo Guardrails (Colang)

```
define user ask about competitors  
    "What about [competitor]?"  
    "Compare with [competitor]"  
  
define flow  
    user ask about competitors  
    bot refuse to discuss competitors  
    bot suggest focusing on our product
```

Guardrail Layers

Input: PII detection, prompt injection, topic restriction

Output: PII redaction, fact verification, toxicity check

Policy: NeMo Guardrails for behavioral constraints

Data Pipeline Operations

RAG Pipeline Ops



Ingestion Monitoring

Track document count, embedding latency, and chunking quality



Index Freshness

Automate re-indexing on document updates; alert on stale indices



Retrieval Quality

Measure recall@k and precision@k continuously



Embedding Versioning

Track which embedding model generated each vector; re-embed on model change

Data Versioning Tools

DVC

Version large datasets
alongside code

Delta Lake / Iceberg

Version tabular data with
time-travel support

MLflow / W&B

Artifact stores for model and data tracking

CI/CD for Agentic AI

```
# .github/workflows/agent-ci.yml
jobs:
  test:
    steps:
      # Unit tests
      - run: pytest tests/unit/ -v

      # Integration tests
      - run: pytest tests/integration/ -v
    env:
      OPENAI_API_KEY: ${secrets.OPENAI_TEST_KEY}

  # Evaluation suite
  - run: python evals/run_evals.py
    --dataset test-set

  # Cost check - fail if over budget
  - run: python evals/check_cost.py
    --max-cost 5.00

  deploy:
    needs: test
    if: github.ref == 'refs/heads/main'
    steps:
      - run: echo "Deploy to production"
```

Pipeline Stages



Unit Tests

Validate individual nodes in isolation



Integration Tests

Full agent flow with test LLM keys



Eval Suite

Accuracy, latency, and cost benchmarks



Deploy

Auto-deploy to production on main branch

Monitoring in Production

Dashboard Metrics

- Request volume and throughput (requests/minute)
- Token consumption (input/output/total per hour)
- Cost tracking (hourly, daily, per-user)
- Error rates by type (LLM error, tool error, timeout)
- Agent path distribution across graph nodes
- User satisfaction signals (thumbs up/down, task completion)

Alerting Rules

Alert	Condition	Severity	Action
Cost spike	> 200% of daily average	High	Investigate & throttle
Error rate surge	> 5% for 5 minutes	Critical	Page on-call engineer
Latency degradation	P95 > 60s for 10 min	Medium	Scale up or check provider
Loop detected	Same node hit 5+ times	High	Kill request, log trace
Token budget exceeded	Project > monthly limit	High	Notify team, throttle

Incident Response

A structured runbook ensures fast, consistent resolution of production incidents.

1

Detect

Automated alert fires via PagerDuty, Opsgenie, or Slack

2

Assess

Check LangSmith traces for failing requests; identify the failing node or tool

3

Mitigate

Switch to fallback model if LLM provider issue; disable failing tool and use fallback logic

4

Fix

Deploy prompt or code fix; run evaluation suite to confirm resolution

5

Post-Mortem

Document root cause, add failing case to eval dataset, update guardrails

Tool and Platform Landscape

Category	Tools	Notes
Tracing / Observability	LangSmith, Langfuse, Phoenix (Arize), Helicone	LangSmith tightly integrated with LangGraph
Prompt Management	LangSmith Hub, PromptLayer, Humanloop	Version, test, and deploy prompts
Evaluation	LangSmith Evals, RAGAS, DeepEval, Braintrust	LLM-as-judge + human-in-the-loop
Guardrails	NeMo Guardrails, Guardrails AI, Lakera	Input/output filtering and policy enforcement
Cost Tracking	Helicone, LiteLLM, OpenMeter	Track per-user and per-project spend
Deployment	Railway, Azure Container Apps, AWS Lambda, Modal	Containerized or serverless deployment
Data Versioning	DVC, MLflow, W&B Artifacts	Track datasets and model artifacts
CI/CD	GitHub Actions, GitLab CI, CircleCI	Automate test + deploy pipelines

You're Ready to Operate Agentic AI in Production

Treat Prompts as Code

Version, test, and deploy with the same rigor as software

Observe Everything

Tracing, metrics, and alerts are non-negotiable in production

Control Costs Early

Cascading, caching, and budget limits prevent runaway spend

Layer Your Safety

Input guards, output guards, and policy enforcement work together

GUIDE 03 — AGENTIC AI BUILDER EXPERT BOOTCAMP

